GeoGuard: A Secure Geo-Tagged Image Authentication System Using Cryptographic Hashing and Digital Signatures

Hemant Mandi, Almas Pathan, Suyog Hanamar, Shubham Hiremath, and Dr. Shaheen Mujawar

Department of Computer Science and Engineering, S. G. Balekundri Institute of Technology, Belagavi, Karnataka, India

Students: Hemant Mandi, Almas Pathan, Suyog Hanamar, Shubham Hiremath; Associate Professor: Dr. Shaheen Mujawar

mandihemant04@gmail.com, almaspathan2006@gmail.com, suyoghanamar21@gmail.com, shubhamhiremath87@gmail.com, shaheenm@sgbit.edu.in

***Abstract*—Photographs are routinely offered as evidence of where and when an event occurred, yet the metadata that carries this claim is easy to edit, and generative models can now produce convincing images of events that never happened. This paper presents GeoGuard, an Android application written in Flutter that makes a photograph self-verifying at the moment of capture. GeoGuard refuses to capture when the operating system flags the location fix as mocked, burns the coordinates, altitude and UTC time into a visible banner, and computes a 64-bit difference hash (dHash) of the banner together with a 4×4 grid of tile hashes over the scene. These values and the device identifier are signed with ECDSA on curve P-256, and the public key is embedded in the file, so any phone can check the signature; a sender is recognised by pinning the fingerprint of its key on first use. The envelope is embedded three times in the file: in the EXIF UserComment field, in a JPEG comment segment, and as an end-of-file annex. Earlier photographs signed with HMAC-SHA256 remain verifiable. At verification the banner and tile hashes are compared with a Hamming tolerance of 10 bits, so that an edit is reported together with the tiles it touched. We state the limits of the design: the private key is held in software and nothing attests that a signature came from a genuine build. Measurements on real photographs show that the 10-bit tolerance is narrow: recompression below JPEG quality 85 can exceed it, and an edit to the date in the banner stayed within it. We also describe end-to-end encrypted photo sharing and an evidence-certificate export laid out after Section 63(4) of the Bharatiya Sakshya Adhiniyam, 2023.**

***Index Terms*—Image authentication, tamper detection, perceptual hashing, dHash, ECDSA, key pinning, mock-location detection, digital evidence, Android.**

I. Introduction

A photograph is accepted as evidence because it appears to bind two claims to a single object: the scene looked like this, and it was recorded at a particular place and time. Insurance adjusters, field inspectors, delivery services and journalists rely on both claims every day, and both have become inexpensive to forge. Free tools rewrite the GPS coordinates and timestamp stored in EXIF metadata in seconds and leave no record that they did so. Splicing, clone-stamping and text replacement change what a scene appears to show, a problem that image forensics has studied for two decades [1], [2]. More recently, generative models such as latent diffusion [3] have made it possible to synthesise photorealistic images of events that never took place, a concern that reaches well beyond the courtroom [4].

The usual response is passive forensics: an analyst examines a finished image for statistical traces of resampling, compression history or sensor noise [1], [5], [6]. Passive methods are indispensable when nothing else is available, but they work after the fact, their evidence weakens as an image is recompressed, and a clean result does not establish where or when a photograph was taken. The alternative is to attach evidence at the moment of capture. The idea goes back to proposals for secure digital cameras [7] and is now standardised for general media by the C2PA specification [8].

Capture-time schemes still leave three practical questions open, and they motivate this work. First, how should a signature bind to pixels? A cryptographic hash over the exact bytes of a file, the hard binding used by C2PA, is invalidated by the first recompression, and messaging applications recompress routinely. Perceptual hashes tolerate such changes [9], [10], but a single hash of a whole frame is too coarse to expose a local edit. Second, how trustworthy is the coordinate being signed? A valid signature over a spoofed position vouches for a lie. Third, how should the outcome be shown to a reader who is not a forensic specialist, and to a court that expects a particular form of certificate for electronic records?

This paper presents GeoGuard, an Android application written in Flutter that addresses these questions in one capture-and-verify workflow. GeoGuard is a working design study, and we are explicit throughout about the guarantees it does and does not give. In particular, the signing key is generated and held in software on the phone rather than in isolated hardware, nothing attests that a signature was made by a genuine build of the application, and a verifier learns who signed a photograph only by pinning the sender's key fingerprint (Section III-D). The contributions are as follows.

1) A capture pipeline that combines a mock-location gate, a visible metadata stamp and signing in a single step, so that no unsigned or spoof-flagged frame is produced by the application.
2) A two-level perceptual binding: a dHash of the stamp banner and a 4×4 grid of tile dHashes over the scene,

all authenticated by the signature. Edits are therefore reported at tile granularity, not only detected.
3) A versioned canonical serialisation (schema versions 3 to 6) that binds the tile hashes and the signing public key, so that stripping a field cannot push the verifier onto a weaker path. The move from HMAC (versions 3 to 5) to ECDSA P-256 (version 6) leaves earlier records verifiable through a legacy key ring.
4) Triple embedding of the envelope (EXIF UserComment, JPEG comment segment, end-of-file annex) with a brace-balanced recovery procedure.
5) Sender attribution by trust-on-first-use pinning of key fingerprints, and end-to-end encrypted photo transfer through a relay that never holds plaintext.
6) An explicit threat model and statement of limits, and a draft electronic-record certificate laid out after Section 63(4) of the Bharatiya Sakshya Adhiniyam, 2023.
7) Measurements on real photographs of how the 10-bit tolerance behaves under recompression, resampling and local edits, including the cases in which it fails.

The remainder of the paper is organised as follows. Section II reviews related work. Section III states the threat model and requirements. Section IV describes the architecture and capture pipeline, Section V the cryptographic and perceptual design, and Section VI the verification pipeline, photo sharing and the evidence certificate. Section VII reports the implementation and evaluation, Section VIII discusses limitations, and Section IX concludes.

II. Related Work

A. Passive Image Forensics

Passive methods search a finished image for inconsistencies. Farid's survey [1] groups them into techniques based on pixel statistics, such as copy-move detection [2], traces of resampling and compression, and camera-based cues such as the sensor pattern noise used for source identification [6]. Verdoliva [5] extends this view to learning-based detectors for manipulated and synthetic media. Passive detectors need no cooperation from the camera, which is their strength, but they give probabilistic evidence, are sensitive to recompression, and say nothing about the place and time of capture. Kee *et al.* examine header-level properties of a JPEG file, such as its quantisation tables, to judge whether it still looks like the output of a camera [11]. GeoGuard is complementary. It does not analyse image statistics; it checks a recorded commitment made at capture.

B. Active Authentication at Capture

Active schemes attach authentication data when the image is created. Blythe and Fridrich [7] argued for building authentication into the camera itself. Semi-fragile watermarking, for example the method of Lin and Chang [12] and its extension to self-recovery [13], was designed to accept JPEG compression while rejecting malicious change, which is the tolerance requirement GeoGuard meets with perceptual hashes. Robust image hashing formalised the competing requirements of robustness to benign processing and security against forgery [9], and Zauner's benchmark [10] compares several perceptual hash functions under common distortions.

GeoGuard uses the difference hash popularised by Krawetz [14], which records the sign of the brightness gradient between horizontally adjacent pixels of a downscaled image; two hashes are compared by their Hamming distance [15]. Its role differs from that in the hashing literature. Here the hash is not the security primitive. It is a value that a signature covers, so an adversary cannot substitute a different hash without the signing key. An adversary may still craft an edit that stays inside the Hamming tolerance, and we treat that as a limit of the scheme (Section III).

C. Provenance Standards and Applications

The C2PA specification [8] defines a manifest of assertions that is signed with an X.509 credential and bound to the asset by cryptographic hashes over its bytes. A validator can therefore confirm that a manifest belongs to exactly this asset, and for the same reason a re-encoded copy no longer validates; the specification also allows optional soft bindings such as watermarks or fingerprints. ProofMode [16] takes a lighter route. It runs alongside the camera, collects contextual metadata, and writes hashes and PGP signatures of the media into separate files, leaving the original untouched; its developers note that it has not yet had a full security audit. Both approaches rely on public-key signatures, so any third party can verify them. GeoGuard's current schema is also public-key, but its trust anchor differs. C2PA chains a signer to a certificate, whereas GeoGuard embeds the signing key in the file, and the verifier must already have pinned the key's fingerprint for the result to name a sender (Section VI). Table I summarises the differences.

D. Key Trust and Secure Messaging

A public key proves nothing about its owner until the verifier has a reason to trust it. Secure-messaging systems face the same problem and answer it in several ways, from certificate authorities to fingerprints compared out of band [17]. GeoGuard takes the lightest route. It uses trust on first use, as SSH does for host keys [18]: the first time a key is accepted it is pinned, and a later photograph is attributed to that sender only if its key matches. The elliptic-curve signature is ECDSA [19], [20] on the curve secp256r1 [21]. Photo transfer between users uses an ephemeral elliptic-curve Diffie–Hellman exchange [22] followed by HKDF [23] (Section VI-B).

E. Location Authenticity

A signed coordinate is only as honest as the sensor that reported it. Humphreys *et al.* built a portable civilian GPS spoofer from commodity components [24], and Tippenhauer *et al.* analysed the conditions under which spoofing attacks against receivers succeed [25]. These are radio-level attacks. A simpler route is to install an application that supplies a mock location through the operating system. Android marks such fixes, and GeoGuard refuses to capture when the flag is set. This closes the easy path and leaves the radio-level one open (adversary A7 in Section III).

TABLE I
DESIGN COMPARISON OF AUTHENTICITY APPROACHES

Approach	Integrity primitive	Binding to content	Edit localisation	Third-party verification
Passive forensics [1], [5]	None (statistical analysis)	Not applicable	Some methods	Yes
Semi-fragile watermarking [12], [13]	Signature embedded in the image	Content features tolerant of JPEG	Yes, block level	Depends on scheme
C2PA [8]	X.509 certificate signature over a manifest	Hard binding: hash of asset bytes (soft binding optional)	Not by itself	Yes
ProofMode [16]	PGP signatures	Hash of the media file	No	Yes
GeoGuard (this work)	ECDSA P-256 (v6); HMAC-SHA256 (v3–v5)	Banner dHash and 16 tile dHashes, tolerance 10 bits	Yes, tile level (one sixteenth of the frame)	Yes; sender named once its key is pinned

F. Legal Context in India

Electronic records are admitted in India through Section 63 of the Bharatiya Sakshya Adhiniyam, 2023 (BSA) [26], which replaced Section 65B of the Indian Evidence Act, 1872. Under the earlier provision the Supreme Court held in *Anvar P. V. v. P. K. Basheer* [27], and restated in *Arjun Panditrao Khotkar v. Kailash Kushanrao Gorantyal* [28], that a certificate is a condition for admitting an electronic record as secondary evidence. Section 63(4) of the BSA requires a certificate in the form given in the Schedule, with one part completed by the person in control of the device and another by an expert. In *Pune Bar Association v. Union of India* [29] a three-judge bench upheld Section 63(4) and described a hash value as an electronic fingerprint that supports checking the integrity of a record. GeoGuard's evidence export (Section VI) pre-fills the technical particulars the application can attest to and leaves both signature blocks blank. We make no claim about admissibility. That is for the court, and certification remains the responsibility of the signatories. This paper is not legal advice.

III. THREAT MODEL AND REQUIREMENTS

A. System and Trust Assumptions

A *capturer* uses GeoGuard on an Android device to photograph a scene. A *verifier* later receives the image file and runs the verification function of the application. We assume that (i) the operating system and the GeoGuard build on the capturing device are unmodified; (ii) the application sandbox keeps its stored key from other applications; (iii) the operating system reports the mock-location flag honestly; and (iv) for schema v6 the verifier learns the sender's key fingerprint through a channel that an attacker cannot rewrite, such as hearing it read aloud, while for schemas v3 to v5 the verifier holds the key ring of the capturing installation. Timestamps are device-reported, and we do not assume a trusted external time source.

B. Adversary Capabilities

Table II lists the adversaries we consider, whether the scheme addresses them, and how. Rows A7 to A9 are out of scope and rows A10 and A11 are covered only in part. We list them because a reader needs to know where the guarantee stops, not only where it holds.

A fully synthetic or regenerated image carries no valid envelope, so the verifier reports it as not signed. An attacker can still sign it with a key of their own, which yields a valid signature from an unknown signer (A10), or with a stolen key (A8). GeoGuard also applies a low-amplitude luminance pattern at capture that is intended to make regenerated images statistically distinguishable. We have not evaluated that pattern, and no security claim in this paper depends on it.

C. Security Objectives

We state six objectives, each of which the verifier reports on separately.

- **O1** *Metadata integrity.* Any change to latitude, longitude, altitude, time, device identifier or the embedded hashes causes verification to fail.
- **O2** *Banner integrity.* A meaningful change to the stamp text is detected, while mild recompression is tolerated.
- **O3** *Scene integrity with localisation.* A substantial edit to the scene is detected and attributed to the tiles it affected.
- **O4** *Position provenance.* No frame is produced from a fix that the operating system flags as mocked.
- **O5** *Durability and honest reporting.* Removing any one metadata location does not destroy the envelope, and the verdict states which checks were actually run, for example that a record predates scene protection.
- **O6** *Sender attribution.* A verifier that has pinned the sender's key fingerprint is told who signed the photograph, and one that has not is told that the signer is unknown.

The verdict is binary. A signature either reproduces or it does not, and a tile is either inside tolerance or outside it, so the report lists the measurements, such as the Hamming distance, the number of matching tiles and the key that verified, instead of a confidence score.

D. Key Custody and Its Consequences

In schema v6 each installation owns one ECDSA key pair on curve P-256 [19]–[21]. A record carries the compressed public key pk (33 bytes, base64), the key identifier $kid = \mathrm{SHA256}(pk)[0{:}16]$ in hexadecimal, and the signature

$$\sigma = \mathrm{ECDSA}_{\mathrm{SHA256}}(d,\, m), \tag{1}$$

TABLE II
ADVERSARY CAPABILITIES AND COVERAGE

ID	Capability	Scope	Mechanism or residual risk
A1	Edits coordinates, altitude, time or device identifier in any embedded copy	In scope	Any change invalidates the signature
A2	Edits the visible stamp banner (text or pixels)	Partly	Banner dHash is signed and a distance above 10 bits is reported, but small text edits can stay inside the tolerance (Section VII)
A3	Splices, clones or removes content in the scene	Partly	16 signed tile hashes; substantial edits caught, an edit small relative to one tile may stay within tolerance
A4	Recompresses the image or strips one metadata location	In scope	Perceptual tolerance; envelope stored in three places; stripping all three yields "not signed"
A5	Removes the tile hashes or key to force a weaker check	In scope	Version, tiles and public key are part of the signed text
A6	Supplies a fake location through a mock-location application	In scope	Capture is refused when the OS flags the fix
A7	Modified OS or firmware, or radio-level GNSS spoofing	Out of scope	Cannot be detected from the application layer
A8	Extracts the private key (rooted device, code running inside the app, restored backup)	Out of scope	Forged records for that identity; no revocation
A9	Photographs a screen or print showing a false scene	Out of scope	The record is a genuine capture of a false scene
A10	Signs a forged photograph with a freshly generated key pair or a re-implemented signer	Partly	The signature verifies, but the key is not pinned, so the report says the photo came from another, unsaved phone (Section III-D)
A11	Substitutes a key in the user directory before a sender is saved	Partly	The directory is trusted once; comparing fingerprints out of band is the real check

where d is the private scalar and m is the canonical serialisation of the envelope fields, which includes pk. The identifier kid is the code that users read to one another and the fingerprint that a verifier pins. Four consequences follow, and we state them because they bound what a verdict means.

1) The key is held in software. It is generated in application code, and the 32-byte private scalar is stored through a secure-storage plugin, which encrypts it with an AES key that is itself wrapped by a key in the Android Keystore [30]. The wrapping key is hardware-backed only if the phone places Keystore keys in secure hardware, and the application neither requests nor checks that. During signing the scalar is decrypted into application memory and handed to a background worker, where signing runs in software. The scheme therefore resists other applications and casual file access, but not code that runs with the application's privileges.
2) Nothing attests the signer. The embedded key proves only that the holder of that key signed the record. It does not show that the key was produced by a genuine build on a real phone, so a re-implemented signer with a fresh key makes valid-looking records (A10). Attribution comes entirely from pinning, as in Section VI.
3) Verification needs no secret. Anyone can check a v6 signature from the embedded key, which removes the main limit of the earlier symmetric scheme, but the check says *which key* signed, not *who* the holder is.
4) The earlier schemes are retained for verification only. Records of schemas v3 to v5 are checked against a local HMAC key ring (Section V-B). Those keys are derived from device identifiers and keep the weaknesses of that design: a symmetric key that can be re-derived by anyone who learns the seed inputs. Because every record also embeds a public device identifier that is a domain-separated hash of the seed, the identifier works as a check value for offline guessing of the 64-bit `ANDROID_ID`. This concerns only legacy keys, since the v6 signing key is independent of the seed.

The scheme therefore protects a record against later tampering by parties who lack the private key, and lets a third party check that protection. It does not protect against a capturer who controls the device or the key. Moving the signing key into hardware with key attestation, and adding revocation, would close the largest gaps and is the principal item of future work.

E. Functional Requirements

From the objectives we derive the requirements that the design must meet. **R1**: the shutter must refuse to fire without a usable GPS fix and must state the reason. **R2**: capture and verification must run on the device without sending photographs or coordinates to a server. The only network traffic of these two functions is the fetching of map tiles; the optional sharing feature uses a server but only for ciphertext (Section VI-B). **R3**: records produced by earlier schema versions must remain verifiable. **R4**: the verifier must report which checks were run and must never imply that a skipped check passed. **R5**: the

interface must remain usable outdoors and with gloves, with touch targets of at least 48 dp.

IV. System Architecture and Capture Pipeline

A. Software Architecture

GeoGuard is a Flutter application for Android. Its source is divided into three layers with a deliberate boundary between them. The *presentation layer* holds screens and view state only. The *service layer* performs all input, output and domain logic: camera control, positioning, device identity, image compositing, signing and verification, certificate generation and the index of captured frames. The *theme layer* holds design tokens and shared interface components. State is managed with plain stateful widgets and a single value notifier for the colour theme, since a state-management framework would add more code than it removes at this size. Cryptographic code lives only in the service layer, which keeps it independent of the interface and open to unit testing.

Fig. 1 shows the capture pipeline that is the core of the application. The account and photo-sharing features visible on the home screen (Fig. 2) are built on top of this pipeline and are described in Section VI. The pipeline has two inputs that must both be valid before any frame is produced, a camera frame and a position fix, and one output, a stamped image whose embedded envelope authenticates it.

B. User Interface

The interface has three tabs, shown in Fig. 2. The *Home* tab opens the camera, starts the photo check, and leads to the sender and received photo lists, and shows the identity of the phone. The *Photos* tab is a grid of the frames that GeoGuard itself has captured, newest first; it never reads the device camera roll. The *Locations* tab plots every captured frame on an interactive OpenStreetMap basemap [31]. A pin is placed only from coordinates read out of the signed envelope inside the frame, so a pin cannot exist without a signed capture behind it. The viewfinder (Fig. 2(b)) shows a status chip with the current fix quality, a preview of the text that will be burned into the next frame, and the shutter. The chip turns to an alert colour and reads "weak" when the reported accuracy is worse than 15 m and "stale" when the fix is older than 30 s, and the shutter refuses to fire and states the reason. Every number that a user might want to check, such as coordinates, accuracy and time, is set in a monospaced face, and touch targets are at least 48 dp so that the application can be used with gloves.

C. Capture Pipeline

Algorithm 1 gives the capture procedure. It has five steps.

Acquisition. The sensor frame, a best-accuracy position fix and the public device identifier are requested concurrently, which keeps shutter lag down. The position request first checks that location services are enabled and fails if they are not, so a frame can never be produced silently without a location.

Gate. If the operating system flags the fix as coming from a mock provider, the capture is refused and no frame is written. Signing a spoofed coordinate would make the application vouch for a false statement, so the gate sits before any image processing.

Algorithm 1 Capture and sign

Require: camera c, device service D, identity store S
1: $(I_{\text{raw}}, g, dev) \leftarrow$ concurrently c.TAKE(), READFIX(), D.PUBLICID()
2: **if** location services are off **or** g is unusable **then**
3: **abort**
4: **end if**
5: **if** $g.isMocked$ **then**
6: **abort** ▷ no frame is produced
7: **end if**
8: $I_s \leftarrow \text{STAMP}(I_{\text{raw}}, g)$ ▷ banner and micro-pattern
9: $h_B \leftarrow \text{DHASH}(\text{CROP}(I_s,\ \text{bottom } 18\%))$
10: $(h_1, \ldots, h_{16}) \leftarrow \text{TILEHASHES}(I_s)$
11: $(d, pk) \leftarrow S.\text{IDENTITY}();\quad kid \leftarrow \text{KID}(pk)$
12: $m \leftarrow \text{CANON}_6(lat, lon, alt, ts, dev, h_B, h_1..h_{16}, pk)$
13: $\sigma \leftarrow \text{ECDSASIGN}(d, m)$ ▷ background worker
14: $E \leftarrow \text{JSON}(\ldots, \sigma, pk, kid, v{=}6)$
15: $bytes \leftarrow \text{EMBED}(I_s, E)$ ▷ EXIF, COM, EOF
16: write file, export to album
17: **return** $bytes$

Stamping. The decoded frame is modified in three ways (Table III, Fig. 3). A luminance micro-pattern is applied across the whole image. A panel is then filled across the bottom 18 % of the frame, and three lines of text are drawn on it: a short location code with the locality, the coordinates in degrees, and the date and time with the UTC offset. A denser dual-tone pattern fills a strip beneath the text. A filled panel is used instead of a drop shadow because text with only a shadow fails on both bright sky and dark interiors. The text size is a fixed fraction of the image height, so the stamp scales with the frame.

Hashing and signing. Both perceptual hashes are computed on the stamped image, the envelope is built and authenticated, and it is embedded as described in Section V.

Persistence. The final bytes are written to the application's temporary directory under a name that contains the capture time in milliseconds, and are exported to a GeoGuard album in the device gallery. The bytes returned by the procedure are final: they are already stamped, signed and embedded, and signing them again would append a second payload and corrupt verification.

V. Cryptographic and Perceptual Design

A. Envelope and Canonical Serialisation

Each capture carries a compact JSON *envelope*. Table IV lists its fields. The envelope is prefixed with a marker so that it can be found by a raw byte scan: `VPIC_METADATA:` for the EXIF and comment copies and `VPIC_PAYLOAD:` for the end-of-file copy. The application was renamed from VeriPic to GeoGuard during development. The identifiers inside the markers, the key derivation and the storage keys deliberately keep the old name, because changing any of them would invalidate every photograph already signed.

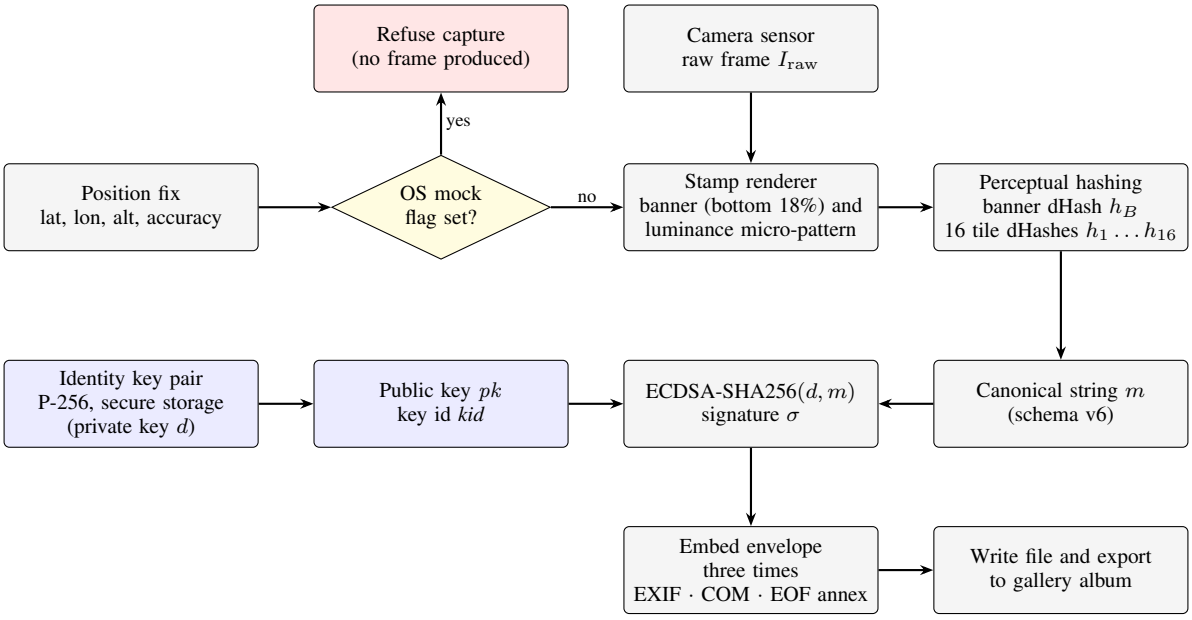
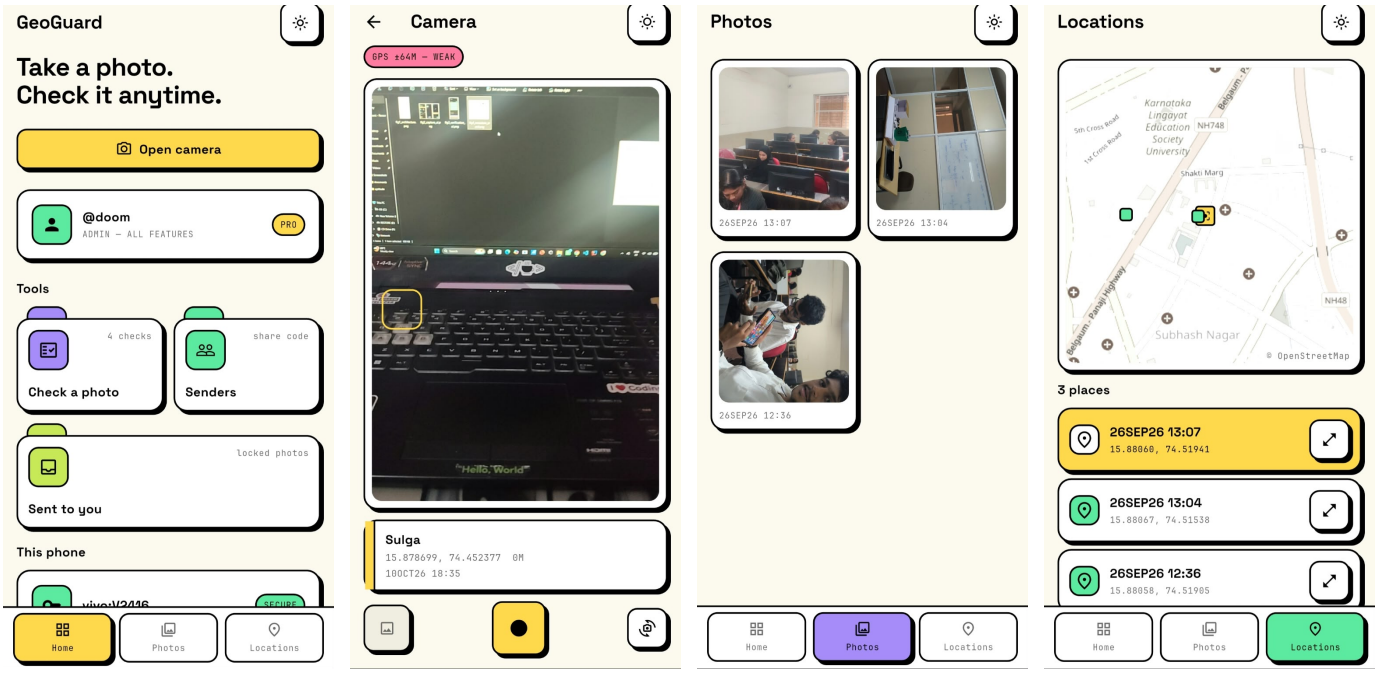


Fig. 1. GeoGuard capture pipeline. A frame is produced only when a position fix is available and the operating system has not flagged it as mocked. The banner and the sixteen scene tiles are hashed after stamping, and the resulting values are signed with the P-256 identity key of the device (Section V).



(a) Home (b) Viewfinder (c) Photos (d) Locations

Fig. 2. GeoGuard application screens. (a) Home tab with the camera, photo check, sender and received-photo entries. (b) Viewfinder with the fix-quality chip (here a weak fix of ±64 m), the preview of the next stamp and the shutter. (c) Grid of captured frames with capture times. (d) Locations tab, showing three signed captures as square pins on an OpenStreetMap basemap, with the selected capture highlighted.

The signature σ is computed not over the JSON text but over a canonical string m whose layout depends on the schema version:

$$m_3 = \texttt{v1} \mid lat \mid lon \mid alt \mid ts \mid dev \mid h_B, \tag{2}$$

$$m_4 = \texttt{v4} \mid lat \mid \cdots \mid h_B \mid kid, \tag{3}$$

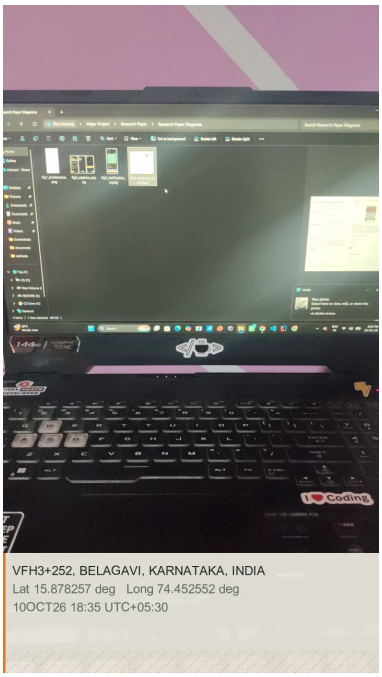
$$m_5 = \texttt{v5} \mid lat \mid \cdots \mid h_B \mid h_1, \dots, h_{16} \mid kid, \tag{4}$$

$$m_6 = \texttt{v6} \mid lat \mid \cdots \mid h_B \mid h_1, \dots, h_{16} \mid pk. \tag{5}$$

Schema 3 and earlier produced the label `v1`. Each later form

TABLE III
STAMPING AND FIX-QUALITY PARAMETERS

Parameter	Value
Banner panel	Bottom 18 % of frame height, cream fill at 92 % opacity
Global micro-pattern	±3 luminance on a 12 px lattice
Banner strip pattern	Dual-tone, full amplitude, below the text
Stamp text	Three lines; size 2.2 % of image height
Fix reported as weak	Accuracy worse than 15 m
Fix reported as stale	Older than 30 s
Geocode refresh	After movement of more than 50 m



(a) Captured frame



(b) Stamp banner (bottom fifth of the frame)

Fig. 3. A frame captured with GeoGuard. The banner holds the location code and locality, the coordinates, and the date and time with the UTC offset. The dotted strip at the bottom edge is the dual-tone pattern.

adds a field to the authenticated string: version 4 the key identifier, version 5 the sixteen tile hashes, and version 6 the public key, with an ECDSA signature in place of the HMAC tag. Because the version label and the tile hashes are inside m, removing the tiles from a v5 or v6 envelope changes m and invalidates σ, and so does removing the key identifier

TABLE IV
ENVELOPE FIELDS (SCHEMA V6)

Field	Type	Content
`lat, lon, alt`	double	Latitude, longitude (degrees), altitude (m)
`ts`	integer	Capture time, epoch ms, UTC
`dev`	string	Public device identifier
`ph`	16 hex	Banner dHash h_B
`st`	16 × 16 hex	Scene tile dHashes $h_1, \ldots, h_{16}$
`pk`	base64	Compressed P-256 public key (33 bytes)
`kid`	16 hex	Key identifier, derived from `pk`
`sig`	hex	ECDSA signature σ (r and s)
`v`	integer	Schema version (currently 6)

from a v4 or v5 envelope; the verifier cannot be pushed onto an older and weaker form by stripping fields. In version 6 the key identifier is derived from the signed public key, so a different key cannot be introduced without a new signature. The test suite covers the stripping cases.

The parser accepts every historical form. A missing version is read as 3, integer coordinates are coerced to doubles, and missing strings default to empty, so the current build reads envelopes written by earlier versions.

B. Legacy HMAC Keys, Key Ring and Device Identity

Schemas 3 to 5 sign with HMAC under a key derived with HKDF-SHA256 [23], [32]. Schema 6 does not use that key to sign (Section III-D), but it remains in the application for checking older records and for the key-diagnostics card on the home screen. With a 32-byte output, which equals the hash length, the expand step has a single iteration, so

$$PRK = \mathrm{HMAC}_{\mathrm{SHA256}}(salt,\ seed), \tag{6}$$

$$K = \mathrm{HMAC}_{\mathrm{SHA256}}(PRK,\ info \,\|\, \texttt{0x01}). \tag{7}$$

The salt and info are fixed strings in the application. The derivation is deterministic, so the key can be regenerated from the seed if secure storage is lost. The key identifier is the first 16 hex characters of $\mathrm{SHA256}(K)$. HMAC provides message authentication under a secret key [33], [34], and is computed over SHA-256 [35].

Device identity is held in three tiers. The *seed* is the secret input to HKDF and never leaves the device or appears in an envelope. It is built from the `ANDROID_ID` value together with the sorted, joined values of the manufacturer, brand, model, device, board and hardware attributes. The build fingerprint is excluded from the seed on purpose: it changes with every over-the-air update, and including it would silently rotate the key and invalidate every earlier photograph on the device. The *public identifier* $dev = \mathrm{SHA256}(dom \,\|\, seed)$, with a fixed domain prefix, is what each envelope carries. The remaining *attributes* are shown to the user for diagnostics. The telephony identifier (IMEI) is not used because ordinary applications cannot read it on current Android versions. If no platform

identifier is available, a random identifier is generated once and stored; the interface flags this case, because records signed under such a key remain verifiable only on that installation.

Rather than a single key, the device holds a *key ring* of these symmetric keys; the v6 identity key pair is stored separately. Each entry has a key identifier, the key material, an origin label and a creation time. The ring exists so that rotating the key after a reset, a device change or a migration never orphans earlier records. Verification tries the key named by *kid* first, then the active key, then every remaining entry, and compares tags in constant time. Records from the earliest schema were signed with a random per-installation secret; that secret is imported into the ring once, marked verify-only, and never used to sign. The legacy key material is the UTF-8 bytes of the stored string, not its base64 decoding, and reproducing this exactly is what keeps old signatures valid. The origin label of the derived key is *hardware-derived* in the source code; because the seed is not held in isolated hardware (Section III-D), this paper calls it *device-derived*.

C. Banner Fingerprint

Re-saving a photograph at another JPEG quality changes every byte, and messaging applications recompress routinely, so a cryptographic hash of the file cannot serve as a binding. GeoGuard instead records a difference hash (dHash) [14], which keeps the direction of brightness gradients rather than absolute values. Recompression barely changes the direction of a gradient, while moving, overwriting or editing text in the banner changes many gradients at once. The banner hash is computed as follows.

1) Crop the bottom 18 % of the stamped image.
2) Resize the crop to 9×8 cells using average interpolation.
3) Convert each cell to luminance $Y = 0.299R+0.587G+0.114B$ [36].
4) For each of the 8 rows, record one bit per adjacent pair of cells, which gives $8 \times 8 = 64$ bits $b_{r,c} = \mathbb{1}[Y_{r,c} > Y_{r,c+1}]$.
5) Serialise the bits as 16 hexadecimal characters.

Two hashes are compared by their Hamming distance [15],

$$d(h, h') = \mathrm{popcount}(h \oplus h'), \tag{8}$$

and the banner is accepted as intact when $d \le \tau = 10$ of 64 bits.

Digests are serialised as *unsigned* 64-bit hexadecimal built from two 32-bit halves. An earlier implementation used a signed integer conversion. For any digest whose top bit was set it produced a 17-character negative string, and the distance routine, which looped while the XOR was positive, returned zero. Two different hashes then compared as identical, a silent false acceptance that affected roughly half of all comparisons. The fault was found and fixed on 22 August 2026. The comparison now runs a fixed 64 iterations and normalises the historical signed spelling, so photographs signed before the fix still compare correctly. Empty inputs, unequal lengths and unparseable hex all return 64, the maximum distance, instead of throwing or matching.

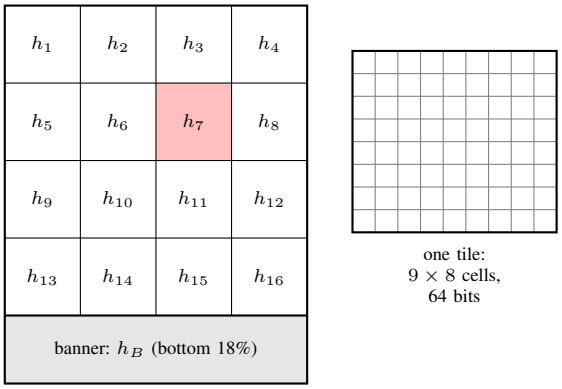


Fig. 4. Scene protection. The top 82 % of the frame is split into sixteen tiles with one 64-bit dHash each, and the bottom 18 % (the banner) has its own hash. A tile is reduced to 9×8 cells, and each hash bit compares two horizontally adjacent cells. An edit that moves only the shaded tile is reported against that tile alone.

D. Tiled Scene Fingerprint

From schema v5 the scene above the banner is fingerprinted too. The top 82 % of the frame is divided into a 4×4 grid, and each of the sixteen tiles receives its own 64-bit dHash with the same tolerance of 10 bits (Fig. 4). A single hash over the whole scene would be too coarse: removing one object barely moves it. Separate tiles keep an edit confined to the tiles it touched, where it stands out, and allow the interface to show *where* the frame changed. All sixteen hashes are inside the signed text (m_5 and m_6, Eqs. 4 and 5), so they cannot be removed without invalidating the signature.

E. What an Edit Must Change to Be Detected

Two observations bound what the tile check can see. First, a hash bit compares two adjacent cells, so one cell takes part in at most two comparisons. To push a tile hash beyond $\tau = 10$ bits an edit must flip at least 11 comparisons, and therefore must alter at least $\lceil 11/2 \rceil = 6$ of the 72 cells of that tile. Each cell is $1/72$ of a tile and a tile is $0.82/16$ of the frame, so a detectable edit must change at least

$$\frac{6}{72} \cdot \frac{0.82}{16} \approx 0.43\,\% \tag{9}$$

of the frame area, and at least $\frac{6}{72} \cdot 0.18 = 1.5\,\%$ for the banner. This is a necessary condition and assumes that each cell depends only on its own block of the resampling; below it, the tile check cannot raise an alarm whatever the content of the edit. Second, if the bits of two unrelated hashes are treated as independent and uniform, the probability that they fall within 10 bits of each other is

$$\sum_{k=0}^{10} \binom{64}{k} 2^{-64} \approx 1.0 \times 10^{-8}. \tag{10}$$

Natural images have strongly correlated gradients, so the true rate for real tiles is higher, and we treat Eq. (10) as a lower bound on chance acceptance, not an estimate of it. The measured behaviour of the tolerance under recompression and under edits is reported in Section VII.

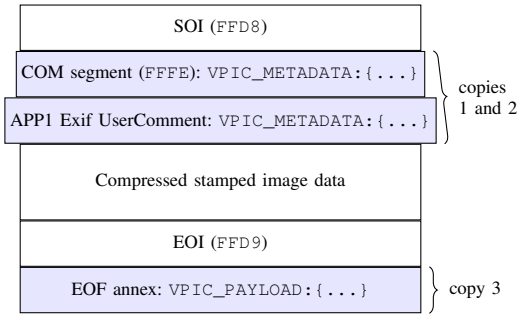


Fig. 5. Byte layout of a signed frame. The envelope appears in the comment segment and in the EXIF UserComment field, both near the start of the file, and again as an annex after the end-of-image marker. Verification reads the annex first and falls back to the other locations.

F. Redundant Triple Embedding

Tools strip different parts of a file, so the envelope is written three times (Fig. 5): in the EXIF UserComment field [37], in a standards-compliant JPEG comment segment with marker `0xFFFE` placed directly after the start-of-image marker [38], and as an annex appended after the end of the compressed data. Decoders ignore the annex, and it survives tools that remove EXIF entirely.

Recovery tries the locations in order of robustness: the annex first, then a raw byte scan for the marker, which finds either the comment or the EXIF copy since both are plain bytes, and finally a decoder-assisted fallback using image text chunks and parsed EXIF. Because the scanned copies have no terminating delimiter, a brace-balanced extractor locates the end of the JSON. It skips braces inside string literals and escape sequences. If no copy is found, the verdict is *not signed*.

VI. Verification, Sharing and Evidence Export

A. Verification Pipeline

The user opens *Check a photo*, picks a file from the gallery or takes a new photo (Fig. 6(a)), and the application runs four checks in sequence while a checklist animates. Algorithm 2 gives the procedure, and Table V lists the verdicts.

Stage 1: find the envelope. Recovery follows Section V-F. If no copy is found the verdict is *not signed*, the remaining stages have nothing to compare, and the screen reads "Not taken with GeoGuard" (Fig. 12(b)).

Stage 2: check the signature. For schema v6 the ECDSA signature is verified against the public key inside the envelope, which needs no secret (Fig. 6(c): "the signature matches — any phone can check this photo"). For schemas v3 to v5 the HMAC is checked against the local key ring: first the key named by *kid*, then the active key, then every other entry, with a constant-time comparison. The report separates a record signed by a key this phone does not hold from one that no key reproduces.

Stage 3: compare the banner. The banner hash is recomputed from the supplied image and compared with the signed value (Eq. 8). The interface shows the distance against the limit as a gauge.

Stage 4: compare the picture. The sixteen tile hashes are recomputed and compared one by one. The result is shown as a 4×4 grid in which each cell carries its own distance (Fig. 6(d)), so an edit is located and not merely detected. Records of schemas v3 and v4 carry no tile hashes. For them the stage reports that it was skipped because the record predates scene protection, and the verdict says so in words, so that a skipped check is never presented as a passed one (requirement R4).

The verdict is binary and carries no percentage. An earlier version blended the checks into one confidence figure; it was removed because a reader could take a failed signature for a strong, partial pass. The report instead lists the measurements: distances, matched tiles and the key that verified. An expandable panel shows every embedded field with tap-to-copy (Fig. 6), and the evidence export described in Section VI-C is one tap away.

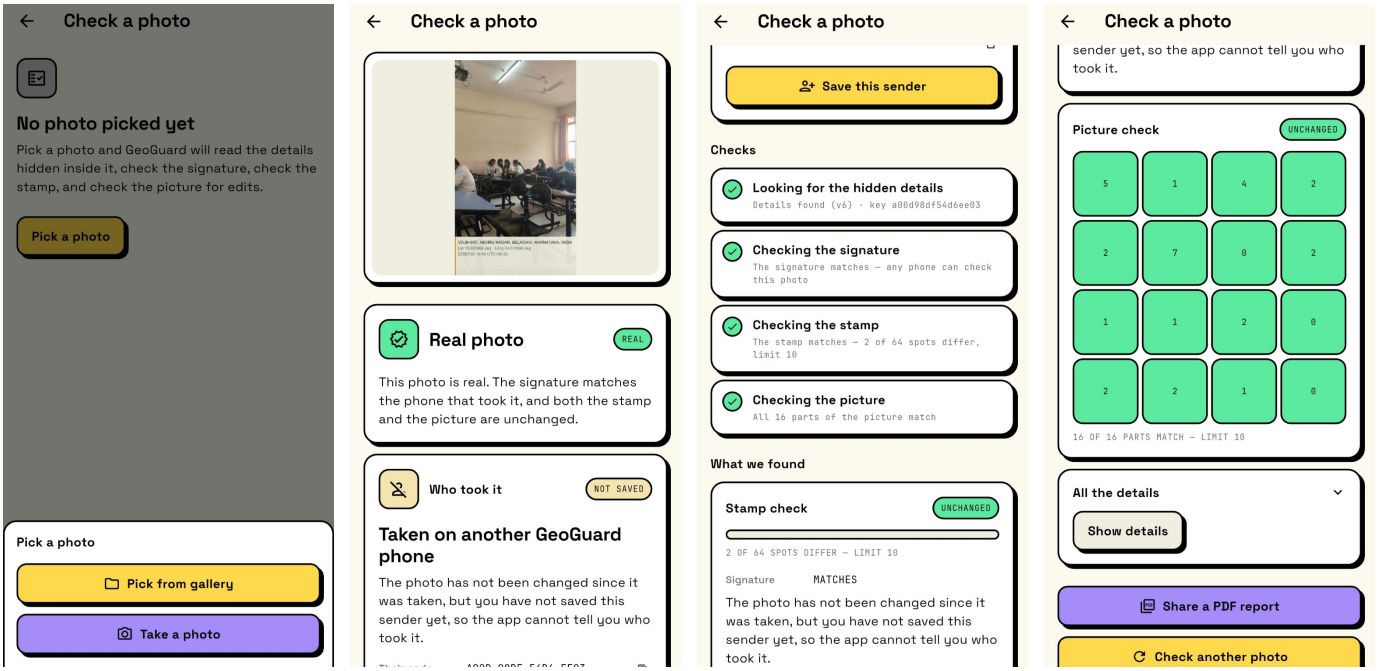
Algorithm 2 Verify

Require: file B, key ring R, pin store P, tolerance τ=10

$E \leftarrow \textsc{Extract}(B)$ ▷ annex, byte scan, decoder fallback
if $E = \perp$ **then**
  return NotSigned
end if
if $E.v \geq 6$ **then**
  $ok \leftarrow \textsc{EcdsaVerify}(E.pk,\ m(E),\ E.\sigma)$
else
  $ok \leftarrow \textsc{HmacVerify}(R,\ m(E),\ E.\sigma)$ ▷ kid, active, rest
end if
$d_B \leftarrow d(\textsc{DHash}(\textsc{Crop}(I,\ \text{bottom } 18\%)),\ E.h_B)$
if $E.v \geq 5$ **then**
  $T \leftarrow \{\, i : d(\textsc{TileHash}_i(I),\ E.h_i) > \tau \,\}$
else
  $T \leftarrow \textsc{Skipped}$
end if
if $\neg ok$ **then**
  return TamperedMetadata
else if $d_B > \tau$ **then**
  return $\textsc{TamperedStamp}(d_B)$
else if $T \neq \emptyset$ **and** $T \neq \textsc{Skipped}$ **then**
  return $\textsc{TamperedPicture}(T)$
end if
$who \leftarrow P.\textsc{Lookup}(\textsc{Kid}(E.pk))$ ▷ v6 only
return $\textsc{Real}(d_B,\ T,\ who)$

TABLE V
Verdicts and Their Conditions

Verdict	Condition
Real	Signature valid, banner within 10 bits, every tile within 10 bits (v5, v6)
Real, scene not covered	As above for v3 and v4, which carry no tile hashes; the report says so
Not signed	No envelope in any of the three locations
Tampered metadata	ECDSA check fails (v6), or no key reproduces the HMAC (v3 to v5)
Tampered stamp	Banner distance above 10 bits
Tampered picture	At least one tile above 10 bits; the tiles are listed



(a) Choose a photo (b) Verdict, sender unsaved (c) Four checks (d) Picture check

Fig. 6. Verification screens. (a) The photo is picked from the gallery or taken with the camera. (b) A genuine photograph received from another phone: the verdict is "real", and the signer is shown by code because that sender has not been saved. (c) The four checks with their measurements, here a banner drift of 2 of 64 bits against a limit of 10. (d) The tile grid; each cell shows its own Hamming distance, and all sixteen are within the limit.

B. Sender Attribution and Photo Sharing

A valid v6 signature says that a key signed the record, not whose key it is. GeoGuard answers the second question by pinning. Every installation has a code, the 16 hexadecimal characters of *kid* shown in groups of four (Fig. 8(c)). The first time a photograph arrives from a sender the application shows the signer's code and offers *Save this sender*. It then stores the fingerprint, a display name and the public key in secure storage, and later photographs whose key has that fingerprint are attributed to the saved name. This is trust on first use [18]. A mismatch is flagged only when a different fingerprint arrives under the same display name, so reading the code aloud, as in secure-messaging practice [17], is the check that actually matters.

Photographs can also be sent inside the application. Signing in uses a Google identity token, which the server validates for signature, audience, expiry and a verified e-mail address. It issues a 30-day session token and keys the account on the Google subject identifier. A user chooses a username and publishes a sharing code, which consists of the signing public key and a separate encryption public key. The directory offers a prefix search that returns at most 20 users and only those who have published a key. For each it returns the username, both public keys and a fingerprint that the server computes itself, and it never returns an e-mail address. Saving a sender found through the directory therefore trusts the directory once.

Transfer is end-to-end encrypted (Fig. 7). The sending phone draws a content key k and an ephemeral key pair (e, E), derives a wrapping key $w = \mathrm{HKDF}(\mathrm{ECDH}(e, R))$ from the recipient's encryption public key R [22], [23], encrypts

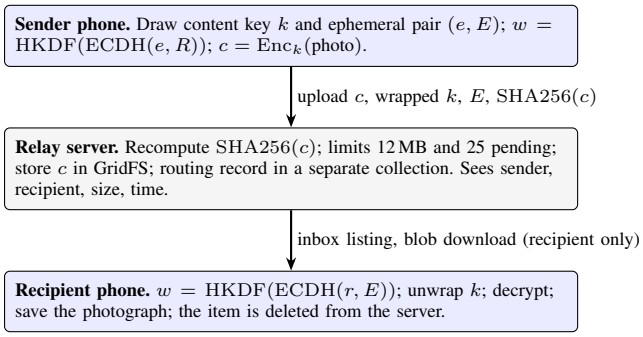
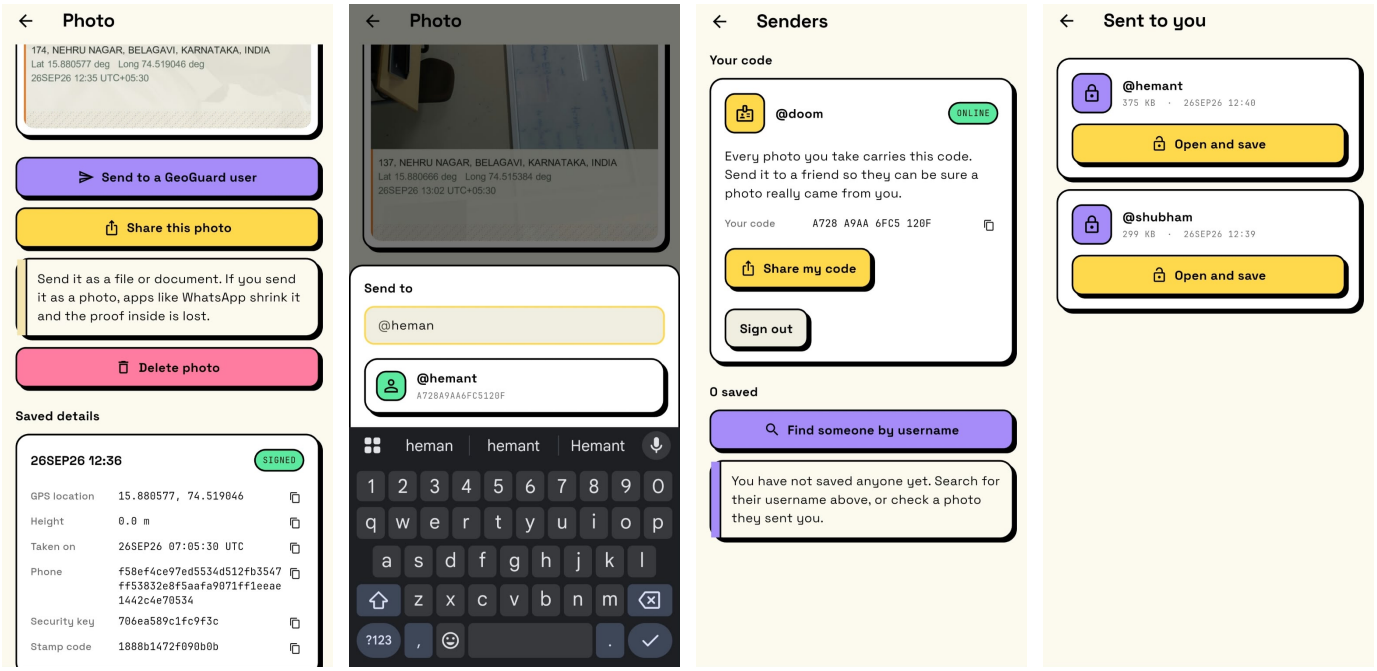


Fig. 7. Sealed photo transfer. The relay stores ciphertext only. R and r are the recipient's public and private encryption keys.

the photograph under k, and uploads the ciphertext c, the wrapped key, E and $\mathrm{SHA256}(c)$. The server recomputes the hash, accepts photographs up to 12 MB, allows at most 25 pending items per recipient, and stores the ciphertext as a blob in a MongoDB GridFS bucket with the routing record in a separate collection. Only the recipient may list, download or delete an item. The application deletes the item after saving the photograph, and the server removes the stored blob at the same moment. The server learns who sent to whom, the size and the time, and nothing of the content.

Sending a photograph through a messaging application as a "photo" lets that application recompress it, which removes the embedded envelope. The photo screen therefore tells the user to send the file as a document (Fig. 8(a)).



(a) Photo screen (b) Send to a user (c) Senders and own code (d) Received items

Fig. 8. Sharing screens. (a) A captured photo with its saved details, the send and share actions, and a warning to send the file as a document. (b) Searching the directory by username shows the fingerprint of each match. (c) The Senders screen shows the user's own code, which is the key fingerprint. (d) Photographs sent to the user are listed as locked items until opened and saved.

C. Evidence Certificate

Any checked file can be exported as a PDF laid out after the certificate that Section 63(4) of the Act [26] requires. It has six numbered paragraphs and two signature blocks (Fig. 9). Paragraph 1 describes the record: type, capture time in UTC, coordinates, altitude, schema, banner fingerprint, the statement that sixteen tile fingerprints cover the picture, and the signature. Paragraph 2 describes the device: model, manufacturer, platform, the public device hash, the identifier source and the key identifier. Paragraph 3 states the manner of production in plain language. Paragraph 4 gives the verification result: verdict, signature scheme, signer public key, banner drift and tile matches. Paragraph 5 shows the frame, and paragraph 6 lists what the certificate does not establish. The declarant's name, designation, address and a case reference are typed in before export, and any field left empty prints as a ruled line. Part A (the person in lawful control of the device) and Part B (a person of expert capacity) are left blank. Unsigned, the document states that it has no legal effect.

The same layout is produced for a file with no envelope. Paragraph 4 then reads "not signed", the signature entries read "not performed", and paragraph 6 adds that the frame predates scene protection or is unverifiable (Fig. 12(c) and (d)). In the build we tested, three passages predate the move to ECDSA: the manner-of-production text and the limits paragraph still describe a key derived from a hardware identifier and a symmetric scheme, and one verdict string still carries the former product name. We list them with the other known issues in Section VIII.

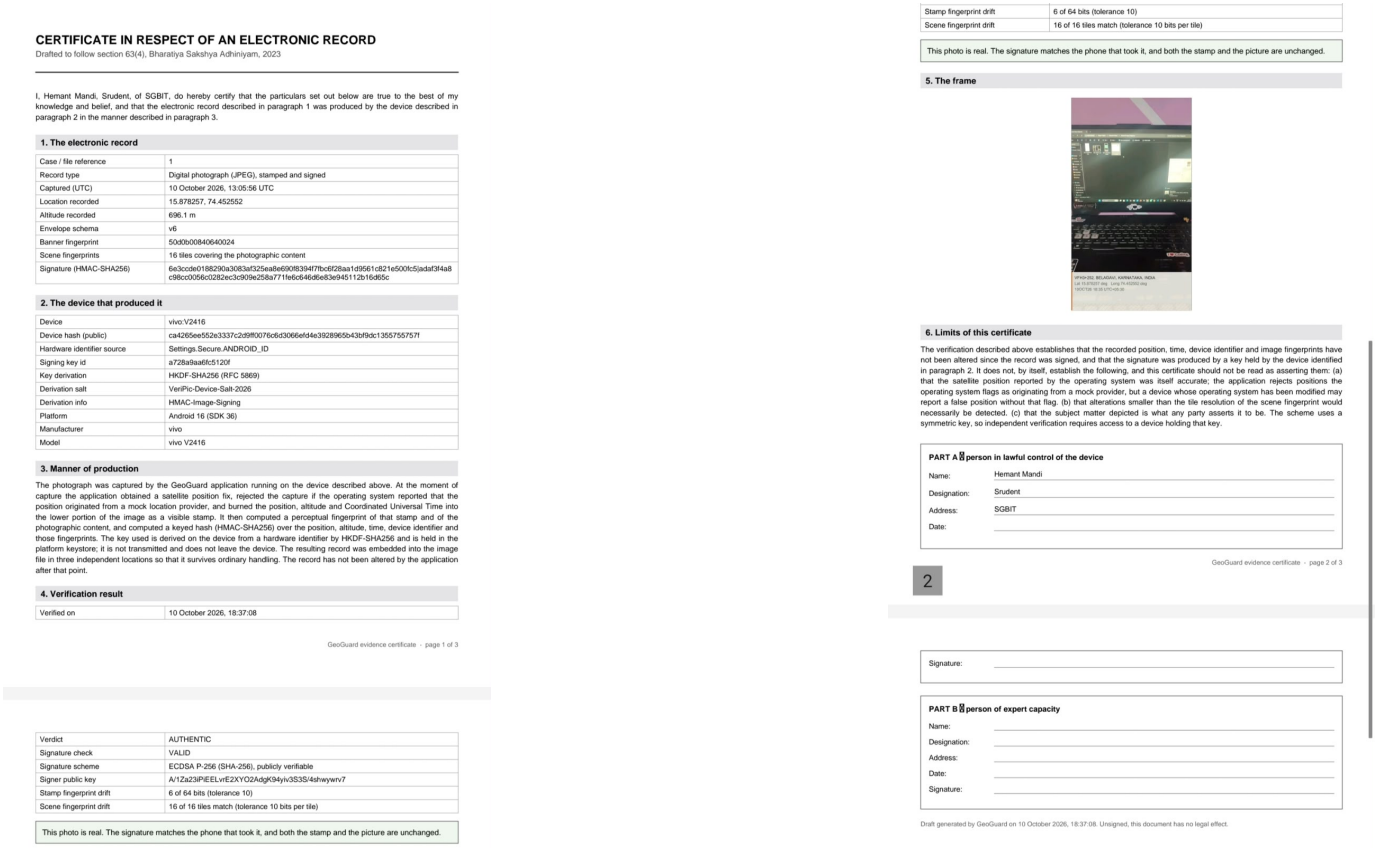
VII. Implementation and Evaluation

A. Implementation

The client is written in Flutter for Android and uses the `camera`, `geolocator`, `image` (decode, composite, encode and EXIF), `pointycastle` (elliptic-curve primitives), `crypto` (SHA-256 and HMAC), `flutter_secure_storage`, `flutter_map` on OpenStreetMap tiles, and `pdf` packages. Signing runs in a background worker so that the interface stays responsive. The sharing service is a small HTTP server backed by MongoDB. All screenshots in this paper were taken on a vivo V2416 phone running Android 16 (API level 36). The repository contains no iOS project, so the application has been built and run for Android only.

B. Test Suite

The client has 52 automated tests in six files and the server has 42 in five files. The client tests cover HKDF against the published RFC 5869 test vectors [23], key identifiers and their serialisation, the Hamming distance including its handling of empty and mismatched input, signing and embedding with recovery from the comment copy after the annex is removed, rejection of an edited envelope, resistance to field stripping, tiling and single-tile localisation, the unsigned-hash serialisation fix, parsing of historical envelopes, sealed transfer, and the frame store. The server tests cover the directory and the sharing endpoints, among others, and skip themselves when no database address is configured. There are no widget or integration tests, so the camera and positioning paths and the interface have not been exercised by automation, and rendering



CERTIFICATE IN RESPECT OF AN ELECTRONIC RECORD

Drafted to follow section 63(4), Bharatiya Sakshya Adhiniyam, 2023

I, Hemant Mandi, Srudent, of SGBIT, do hereby certify that the particulars set out below are true to the best of my knowledge and belief, and that the electronic record described in paragraph 1 was produced by the device described in paragraph 2 in the manner described in paragraph 3.

1. The electronic record

Case / file reference	1
Record type	Digital photograph (JPEG), stamped and signed
Captured (UTC)	10 October 2026, 13:05:56 UTC
Location recorded	15.878257, 74.452552
Altitude recorded	696.1 m
Envelope schema	v6
Banner fingerprint	50d0b00840640024
Scene fingerprints	16 tiles covering the photographic content
Signature (HMAC-SHA256)	6e3ccde0188290a3083af325ea8e690f8394f7fbc6f28aa1d9561c821e500fc5jadaf3f4a8 c98cc0056c0282ec3c909e258a771fe6c646d6e83e945112b16d65c

2. The device that produced it

Device	vivo:V2416
Device hash (public)	ca4265ee552e3337c2d9ff00076c6d3066efd4e3928965b43bf9dc1355755575f
Hardware identifier source	Settings.Secure.ANDROID_ID
Signing key id	a728a9aa6fc5120f
Key derivation	HKDF-SHA256 (RFC 5869)
Derivation salt	VeriPic-Device-Salt-2026
Derivation info	HMAC-Image-Signing
Platform	Android 16 (SDK 36)
Manufacturer	vivo
Model	vivo V2416

3. Manner of production

The photograph was captured by the GeoGuard application running on the device described above. At the moment of capture the application obtained a satellite position fix, rejected the capture if the operating system reported that the position originated from a mock location provider, and burned the position, altitude and Coordinated Universal Time into the lower portion of the image as a visible stamp. It then computed a perceptual fingerprint of that stamp and of the photographic content, and computed a keyed hash (HMAC-SHA256) over the position, altitude, time, device identifier and those fingerprints. The key used is derived on the device from a hardware identifier by HKDF-SHA256 and is held in the platform keystore; it is not transmitted and does not leave the device. The resulting record was embedded into the image file in three independent locations so that it survives ordinary handling. The record has not been altered by the application after that point.

4. Verification result

Verified on	10 October 2026, 18:37:08

GeoGuard evidence certificate · page 1 of 3

Verdict	AUTHENTIC
Signature check	VALID
Signature scheme	ECDSA P-256 (SHA-256), publicly verifiable
Signer public key	A/1Za23lPiEELvrE2XYO2AdgKl94yiv3S3S/4shwywrv7
Stamp fingerprint drift	6 of 64 bits (tolerance 10)
Scene fingerprint drift	16 of 16 tiles match (tolerance 10 bits per tile)

This photo is real. The signature matches the phone that took it, and both the stamp and the picture are unchanged.

Stamp fingerprint drift	6 of 64 bits (tolerance 10)
Scene fingerprint drift	16 of 16 tiles match (tolerance 10 bits per tile)

This photo is real. The signature matches the phone that took it, and both the stamp and the picture are unchanged.

5. The frame

6. Limits of this certificate

The verification described above establishes that the recorded position, time, device identifier and image fingerprints have not been altered since the record was signed, and that the signature was produced by a key held by the device identified in paragraph 2. It does not, by itself, establish the following, and this certificate should not be read as asserting them: (a) that the satellite position reported by the operating system was itself accurate; the application rejects positions the operating system flags as originating from a mock provider, but a device whose operating system has been modified may report a false position without that flag. (b) that alterations smaller than the tile resolution of the scene fingerprint would necessarily be detected. (c) that the subject matter depicted is what any party asserts it to be. The scheme uses a symmetric key, so independent verification requires access to a device holding that key.

PART A — person in lawful control of the device

Name: Hemant Mandi

Designation: Srudent

Address: SGBIT

Date:

GeoGuard evidence certificate · page 2 of 3

2

Signature:

PART B — person of expert capacity

Name:

Designation:

Address:

Date:

Signature:

Draft generated by GeoGuard on 10 October 2026, 18:37:08. Unsigned, this document has no legal effect.

(a) Page 1: record, device, manner of production, result

(b) Page 2: frame, limits and signature blocks

Fig. 9. Evidence certificate for an authentic photograph. The layout follows the six numbered paragraphs of Section VI-C; the declarant and signature blocks are completed by hand. Text is not legible at this size and the figure shows structure only.

at a width of 320 dp with 200% font scale has not been checked.

C. Method of the Hash Experiments

The experiments below ask how the 10-bit tolerance behaves. They are run on a Python re-implementation of the hashing steps in Section V-C and V-D, not on the application itself. The re-implementation uses box averaging to 9×8 cells, BT.601 luminance and the bit rule $\mathbb{1}[Y_{r,c} > Y_{r,c+1}]$, and its resampling filter and bit order may differ from the application's, so absolute distances could differ by a few bits. Two stamped frames were used: a 720×1280 frame of a laptop scene (frame A) and a 280×492 frame of a classroom recovered from an application screenshot (frame B). With two frames the numbers illustrate behaviour and are not rate estimates. Every test image was re-saved as JPEG before hashing, as it would be in use.

D. Recompression and Resampling

Fig. 10 shows the largest tile distance and the banner distance of an *unedited* frame after re-saving at lower JPEG quality. Even at quality 95 the largest tile distance is 5 to 6 bits, so recompression noise uses more than half of the 10-bit budget before any edit is made. Up to quality 85 all distances stay at 9 or below. Below that the largest tile distance exceeds 10 on both frames (at quality 70 on A and at 80 on B), and the banner exceeds 10 on A at quality 80. A genuine photograph recompressed at those qualities would therefore be reported as altered. Downscaling to between 35 % and 90 % of the linear size and saving at quality 85 gave a largest tile distance of 6 to 11, with one case above the limit. The tolerance is thus adequate for light recompression and unreliable for the stronger recompression that some messaging services apply.

E. Detecting Local Edits

We placed a patch of a given area at a random position in the scene, saved the result at quality 90, and counted how often some tile exceeded 10 bits. Three edits were used: pasting a patch copied from elsewhere in the same frame, filling the patch with its mean colour, and blurring it with a Gaussian of $\sigma = 6$ px. There were 200 trials per setting and frame, 400 in all per point in Fig. 11. A pasted patch is caught in half of the trials at about 2 % of the frame area and in nearly all at 5 %. At 0.25 to 0.5 %, the region around the necessary bound of Eq. (9), only about 5 % of pasted patches are caught. A flat fill needs somewhat more area. A blur is barely noticed: even at 12 % of the frame it was caught in 41 % of trials. This agrees with the scheme's design, which records the sign of brightness gradients between cells and so ignores smoothing.

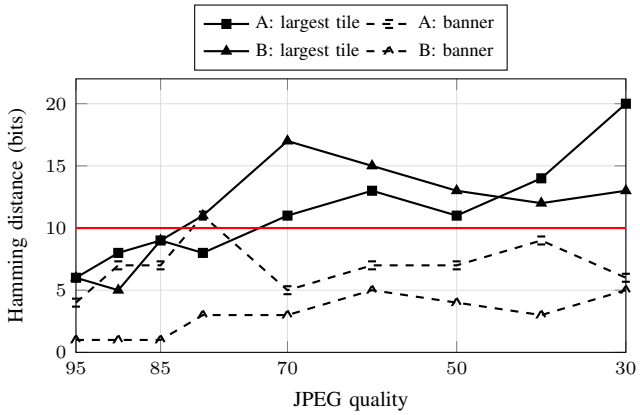


Fig. 10. Distance between an unedited frame and its recompressed copy, for the largest tile and for the banner. The horizontal line is the tolerance of 10 bits.

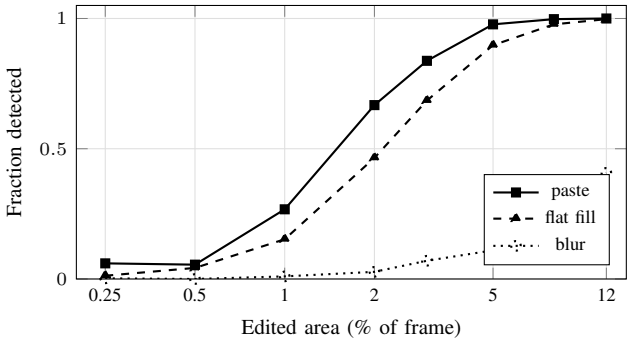


Fig. 11. Fraction of local edits detected by the tile check, as a function of the edited area, averaged over two frames (400 trials per point). Detection needs roughly 2 % of the frame for a paste and far more for a blur.

It also means that blurring a face or a licence plate is an edit that the picture check will usually miss.

For unrelated content the picture is reassuring. Over all 496 pairs of tiles from the two frames the smallest distance was 14 bits, the mean was 31.6, and no pair fell within 10 bits. This is consistent with the lower bound of Eq. (10), although 496 pairs cannot resolve a probability near 10^{-8}.

F. Editing the Banner

The banner hash is meant to expose edits to the stamp text, and Table VI shows how it does. The first row is the baseline: an untouched banner saved at quality 90 already differs by 7 bits, because the dual-tone strip at the bottom of the banner is a fine pattern that JPEG alters. Erasing the date, or replacing it, leaves the distance at 6 to 8 bits. Only when a whole line of text is erased (about 8 % of the banner) does the distance reach 11 or 12, and erasing all the text gives 17. Our real edited photograph confirms this. Fig. 12(a) is the original frame after an image-editing tool changed the date from 10OCT26 to 12OCT26 in a different face. Its banner distance from the original is 7 bits and no tile moves by more than 5. The edit tool also dropped the envelope, so the application reported the file as not taken with GeoGuard (Fig. 12(b)). Had the envelope survived, the hash check would have passed, and the report would have shown the true capture time from the envelope beside a banner that says otherwise. The procedure compares hashes and does not read the banner text.

TABLE VI
BANNER DISTANCE AFTER EDITS (RE-IMPLEMENTATION, FRAME A, JPEG 90)

Edit	Banner area	Distance
None (recompression only)	0 %	7
Erase the date token	1.9 %	6
Erase the date line	5.3 %	8
Erase the coordinates line	8.2 %	11
Erase the locality line	8.5 %	12
Erase all text	47 %	17
Real edit, date 10 to 12	small	7

G. Observations on the Device

Two verification reports taken on the phone give the real behaviour of the application. For a photograph captured and checked on the same phone, the banner differed from the signed hash by 6 of 64 bits and all sixteen tiles were within tolerance (Fig. 9). For a photograph received from another phone the banner differed by 2 bits, and the sixteen tile distances were 5, 1, 4, 2, 2, 7, 0, 2, 1, 1, 2, 0, 2, 2, 1 and 0, a mean of 2.0 and a maximum of 7 (Fig. 6(d)). Neither photograph was edited, so these are the residual drift of the pipeline itself. We attribute it to JPEG encoding after the hashes are computed, but we did not isolate the cause. The largest value, 7 of a limit of 10, matches the recompression experiment: the margin is real but small.

H. Envelope Overhead

A representative v6 envelope, built from the fields of Table IV with random values, is 693 bytes of JSON. Three copies add about 2.1 kB to a file, which is under 1 % of the 299 to 375 kB photographs listed in Fig. 8(d).

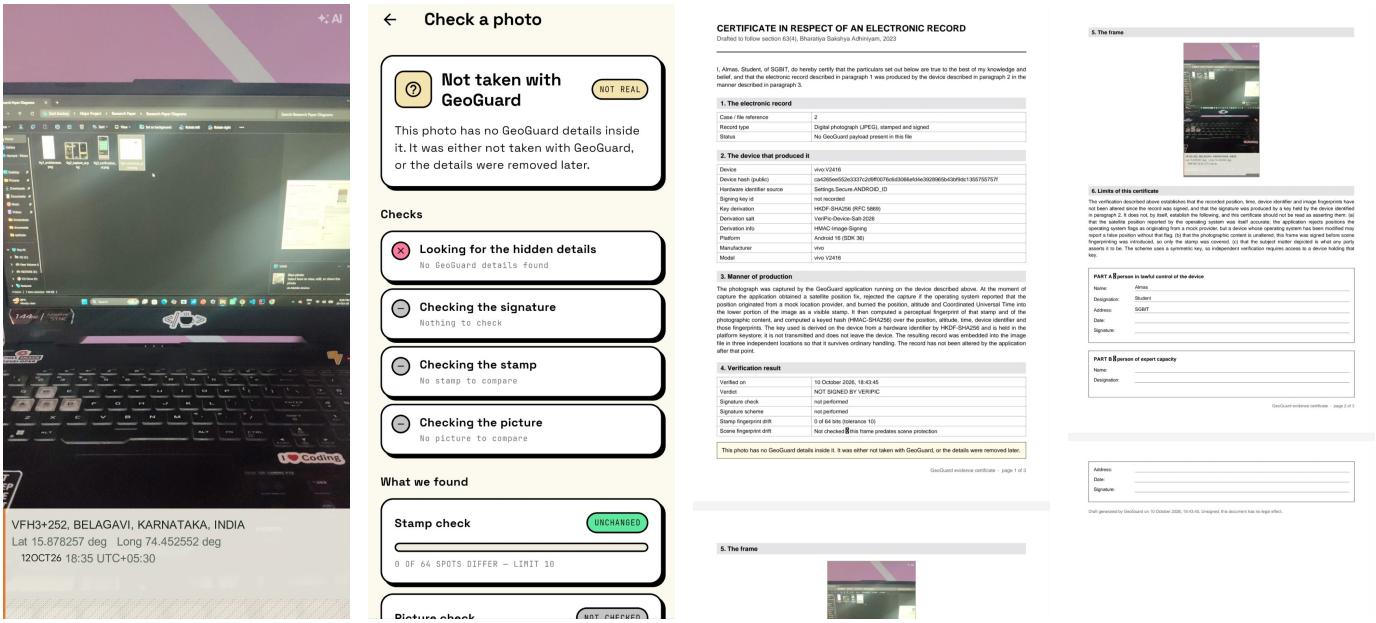
VIII. DISCUSSION AND LIMITATIONS

A. What a Verdict Means

A *real* verdict for a v6 photograph means four things. The record was signed by the holder of the embedded key. Its coordinates, altitude, time and device identifier have not changed since. The banner and the sixteen tiles are within 10 bits of the signed hashes. And the capture was not refused by the mock-location gate. If the sender's code has been pinned, the verdict also names the sender. It does not mean that the place and time are true, that the scene is not staged, that no small edit was made, or that the signing key came from a genuine copy of the application.

B. Limitations

The tolerance is narrow on both sides. Genuine photographs recompressed below JPEG quality 85 can be flagged, while a



(a) Edited frame (b) Verdict (c) Certificate, page 1 (d) Certificate, page 2

Fig. 12. An edited photograph. (a) The date in the banner was changed from 10OCT26 to 12OCT26 with an image editor. (b) The editor removed the envelope, so the application reports that the photo was not taken with GeoGuard. (c, d) The certificate states "not signed" and that the signature check was not performed.

pasted patch covering 1 % of the frame was caught in only about a quarter of the trials and a date edit in the banner passed. A single fixed threshold of 10 bits cannot serve both purposes well.

The banner text is weakly bound. A change to the date or coordinates in the visible stamp can stay within tolerance, and the verifier does not compare the stamp text with the signed fields. Blurring is largely invisible to the picture check.

The key is software-held and unattested. Section III-D explains why a re-implemented signer can produce valid-looking records, why attribution depends on pinning, and why a stolen key allows forgery with no revocation. Because the application does not set the Android backup flag, the platform default applies, so application data may be included in device backups. The stored private key is encrypted, and the Keystore wrapping key is not part of a backup, according to our reading of the code.

Pinning is weak at the first step. Saving a sender from the directory trusts the server once, and the conflict warning compares display names, not keys.

Position and time are device-reported. Only fixes that the operating system marks as mocked are refused. A modified system or radio-level spoofing is not detected, and there is no trusted time source. Coordinates are not written to the EXIF GPS tags, and the unstamped original frame is not kept.

Known issues in the tested build. Two parts of the certificate text predate ECDSA, one verdict string still shows the former name, and the stamp panel of an unsigned file reads "0 of 64 differ" although nothing was compared, which contradicts requirement R4. The stamp text is drawn with a built-in bitmap face and not with the design typeface. Map tiles are not cached for offline use.

Not evaluated. The luminance micro-pattern against generated images, usability with real users, behaviour on other phone models, and the application on iOS. The hash experiments use two frames and a re-implementation.

Legal. The certificate is a draft. We make no claim about admissibility.

C. Future Work

The most valuable step is to generate the signing key inside the Android Keystore, so that it cannot be exported, and to use key attestation so that a verifier can see that the key lives in hardware [30]. A revocation or key-transparency mechanism would then address key loss and directory substitution. The banner can be bound more tightly by re-rendering it from the signed fields and comparing, or by comparing the recovered text. The tolerance could depend on the measured recompression level or move to a finer tile grid. Exporting a C2PA manifest [8] would let existing validators read GeoGuard records. Caching map tiles and writing EXIF GPS tags are smaller items.

IX. Conclusion

GeoGuard turns a phone camera into a signer of geo-tagged photographs. It refuses mocked positions, stamps the coordinates and time into the frame, fingerprints the banner and sixteen scene tiles with difference hashes, signs the result with ECDSA P-256, and stores the signed envelope in three places in the file. Any phone can check such a signature, a sender is recognised by a pinned key fingerprint, and photographs can be sent end to end encrypted. The measurements show where the design is weak: the 10-bit tolerance is inside the

recompression noise at low JPEG quality, small edits and blurs pass, and a change to the banner date went unnoticed by the hash. The key is held in software and carries no attestation. We presented these limits next to the guarantees, because a verdict is useful only to a reader who knows what it does not show. A hardware-backed attested key and a tighter binding of the stamp text are the next steps.

REFERENCES

[1] H. Farid, "Image forgery detection," *IEEE Signal Process. Mag.*, vol. 26, no. 2, pp. 16–25, Mar. 2009.

[2] J. Fridrich, D. Soukal, and J. Lukáš, "Detection of copy-move forgery in digital images," in *Proc. Digit. Forensic Res. Workshop (DFRWS)*, Cleveland, OH, USA, Aug. 2003.

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 10 684–10 695.

[4] R. Chesney and D. Citron, "Deep fakes: A looming challenge for privacy, democracy, and national security," *Calif. Law Rev.*, vol. 107, p. 1753, 2019.

[5] L. Verdoliva, "Media forensics and DeepFakes: An overview," *IEEE J. Sel. Topics Signal Process.*, vol. 14, no. 5, pp. 910–932, Aug. 2020.

[6] J. Lukáš, J. Fridrich, and M. Goljan, "Digital camera identification from sensor pattern noise," *IEEE Trans. Inf. Forensics Security*, vol. 1, no. 2, pp. 205–214, Jun. 2006.

[7] P. Blythe and J. Fridrich, "Secure digital camera," in *Proc. Digit. Forensic Res. Workshop (DFRWS)*, Baltimore, MD, USA, Aug. 2004.

[8] Coalition for Content Provenance and Authenticity, "C2PA technical specification, version 2.3," C2PA, Tech. Rep., Dec. 2025, [Online]. Available: https://spec.c2pa.org/specifications/specifications/2.3/specs/C2PA_Specification.

[9] A. Swaminathan, Y. Mao, and M. Wu, "Robust and secure image hashing," *IEEE Trans. Inf. Forensics Security*, vol. 1, no. 2, pp. 215–230, Jun. 2006.

[10] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," Master's thesis, Univ. Appl. Sci. Upper Austria, Secure Inf. Syst. Program, Hagenberg, Austria, Jul. 2010.

[11] E. Kee, M. K. Johnson, and H. Farid, "Digital image authentication from JPEG headers," *IEEE Trans. Inf. Forensics Security*, 2011.

[12] C.-Y. Lin and S.-F. Chang, "A robust image authentication method surviving JPEG lossy compression," in *Proc. SPIE, Storage and Retrieval for Image and Video Databases VI*, vol. 3312, San Jose, CA, USA, Jan. 1998.

[13] ——, "SARI: Self-authentication-and-recovery image watermarking system," in *Proc. ACM Int. Conf. Multimedia*, Ottawa, ON, Canada, 2001.

[14] N. Krawetz, "Kind of like that," The Hacker Factor Blog, 2013. [Online]. Available: http://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html.

[15] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, Apr. 1950.

[16] Guardian Project, "ProofMode: Verified visuals," [Online]. Available: https://guardianproject.info/apps/org.witness.proofmode/.

[17] N. Unger, S. Dechand, J. Bonneau, S. Fahl, H. Perl, I. Goldberg, and M. Smith, "SoK: Secure messaging," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Jose, CA, USA, May 2015.

[18] D. Wendlandt, D. G. Andersen, and A. Perrig, "Perspectives: Improving SSH-style host authentication with multi-path probing," in *Proc. USENIX Annu. Tech. Conf.*, Boston, MA, USA, Jun. 2008.

[19] D. Johnson, A. Menezes, and S. Vanstone, "The elliptic curve digital signature algorithm (ECDSA)," *Int. J. Inf. Secur.*, vol. 1, pp. 36–63, 2001.

[20] National Institute of Standards and Technology, "Digital signature standard (DSS)," NIST, FIPS PUB 186-5, Feb. 2023.

[21] Standards for Efficient Cryptography Group, "SEC 2: Recommended elliptic curve domain parameters, version 2.0," Certicom Research, Tech. Rep., Jan. 2010.

[22] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, "Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography," NIST, Special Publication 800-56A Rev. 3, Apr. 2018.

[23] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," IETF, RFC 5869, May 2010.

[24] T. E. Humphreys, B. M. Ledvina, M. L. Psiaki, B. W. O'Hanlon, and J. P. M. Kintner, "Assessing the spoofing threat: Development of a portable GPS civilian spoofer," in *Proc. ION GNSS Int. Tech. Meeting Satell. Div.*, Savannah, GA, USA, Sep. 2008.

[25] N. O. Tippenhauer, C. Pöpper, K. B. Rasmussen, and S. Čapkun, "On the requirements for successful GPS spoofing attacks," in *Proc. 18th ACM Conf. Comput. Commun. Secur. (CCS)*, Chicago, IL, USA, 2011.

[26] "The Bharatiya Sakshya Adhiniyam, 2023 (Act No. 47 of 2023)," Gazette of India, 2023.

[27] "Anvar P. V. v. P. K. Basheer," (2014) 10 SCC 473 (India).

[28] "Arjun Panditrao Khotkar v. Kailash Kushanrao Gorantyal," (2020) 7 SCC 1 (India).

[29] "Pune Bar Association v. Union of India," 2026 SCC OnLine SC 1297 (India), decided May 22, 2026.

[30] Android Developers, "Android Keystore system," [Online]. Available: https://developer.android.com/privacy-and-security/keystore.

[31] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," *IEEE Pervasive Comput.*, vol. 7, no. 4, pp. 12–18, Oct. 2008.

[32] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme," in *Advances in Cryptology—CRYPTO 2010*, ser. Lecture Notes in Computer Science, vol. 6223. Berlin, Germany: Springer, 2010.

[33] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF, RFC 2104, Feb. 1997.

[34] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in *Advances in Cryptology—CRYPTO '96*, ser. Lecture Notes in Computer Science, vol. 1109. Berlin, Germany: Springer, 1996.

[35] National Institute of Standards and Technology, "Secure hash standard (SHS)," NIST, FIPS PUB 180-4, Aug. 2015.

[36] International Telecommunication Union, "Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," ITU-R, Recommendation BT.601-7, Mar. 2011.

[37] Camera & Imaging Products Association, "Exchangeable image file format for digital still cameras: Exif version 2.31," CIPA, Standard DC-008-2016, 2016.

[38] International Telecommunication Union, "Information technology—digital compression and coding of continuous-tone still images—requirements and guidelines," ITU-T, Recommendation T.81, Sep. 1992.